Universidad Nacional Autónoma de México

Facultad de Ingeniería

Laboratorio de Arquitectura y Organización de Computadoras

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 6

Secuenciador básico de 4 microinstrucciones

Fecha de Entrega

16 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Familiarizar al alumno en el conocimiento del secuenciador básico, el cual es una parte fundamental del procesador.

2. Desarrollo

Un secuenciador es el bloque encargado de generar la dirección de la siguiente microinstrucción. En una unidad de control microprogramada, el registro mPC contiene la dirección actual de la memoria de control. A partir de ella, el secuenciador puede continuar con la dirección consecutiva, cargar una liga asociada a un salto condicional, utilizar una dirección de transformación o atender una interrupción.

Para esta práctica se seleccionó la alternativa de cuatro microinstrucciones. El diseño se realizó con un contador de microprograma de cuatro bits, un incrementador, tres registros que entregan direcciones fijas y un multiplexor de cuatro entradas. La lógica interna recibe los bits $I_1 I_0$ de la microinstrucción y la condición de control CC, con lo que genera los dos bits de selección del multiplexor.

2.1. Microinstrucciones del secuenciador

En el esquema proporcionado se considera una secuenciador alternativo donde se utiliza una ROM que entrega los campos de prueba, valor de prueba, tipo de microinstrucción, liga y salidas. En esta alternativa se implementó directamente el mecanismo de selección. Para probarlo, se aplicaron I_1, I_0 y CC como entradas externas, los registros de liga, transformación e interrupción representan las direcciones almacenadas que normalmente provendrían de los campos correspondientes. La Figura[1] muestra el diagrama del secuenciador básico alternativo proporcionado.

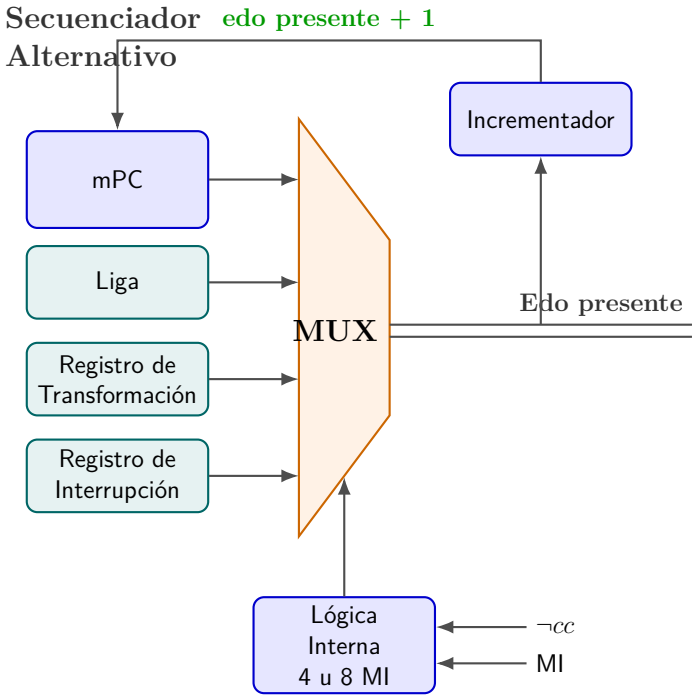


Figura 1: Diagrama del secuenciador básico alternativo.

Las cuatro opciones se definen con los bits $I_1 I_0$ (MI). El paso continuo selecciona el valor $mPC + 1$. El salto condicional selecciona la liga solamente cuando $CC = 0$. El salto de transformación usa el registro RT sin depender de CC. Finalmente, la microinstrucción de interrupción selecciona el registro RI cuando $CC = 0$, donde si la condición no solicita la interrupción, el contador continúa normalmente.

Tabla 1: Tabla de operación del secuenciador básico alternativo.

I_1	I_0	CC	`sel`	Operación realizada
0	0	X	00	Paso continuo: $mPC \leftarrow mPC + 1$.
0	1	0	11	Salto condicional: $mPC \leftarrow$ Liga.
0	1	1	00	No se cumple el salto: $mPC \leftarrow mPC + 1$.
1	0	X	01	Salto de transformación: $mPC \leftarrow RT$.
1	1	0	10	Salto de interrupción: $mPC \leftarrow RI$.
1	1	1	00	No hay interrupción: $mPC \leftarrow mPC + 1$.

Para la implementación se usaron las direcciones fijas de la Tabla[2]. Estos valores permiten distinguir de manera más clara cada fuente cuando se observa la salida del contador de microprograma. El incremento se obtiene de manera combinacional a partir de la dirección presente.

Tabla 2: Fuentes de dirección utilizadas en la implementación.

Fuente	Dirección de prueba	Selección del multiplexor
Paso continuo	$mPC + 1$	00
Registro de liga	`1000`	11
Registro de transformación (RT)	`1010`	01
Registro de interrupción (RI)	`1100`	10

2.2. Arquitectura implementada

El registro mPC conserva la microdirección presente y se actualiza únicamente en el flanco ascendente del reloj. Su salida se realimenta al incrementador, el cual genera $mPC + 1$. El multiplexor recibe cuatro posibles direcciones: la consecutiva, la liga, RT y RI. La lógica interna entrega la señal `sel[1..0]` que determina cuál de ellas llegará al registro mPC en el siguiente pulso.

La Figura[2] muestra el diagrama de bloques de conexión final de los módulos. La señal de reloj tiene un divisor de frecuencia mediante el bloque `clk_buttom`, por cada flanco de subida se avanza una microinstrucción a la vez desde el botón de la tarjeta. Las entradas I_0, I_1 y CC alimentan la lógica interna, donde la salida `edo_pr[3..0]` permite observar la microdirección presente.

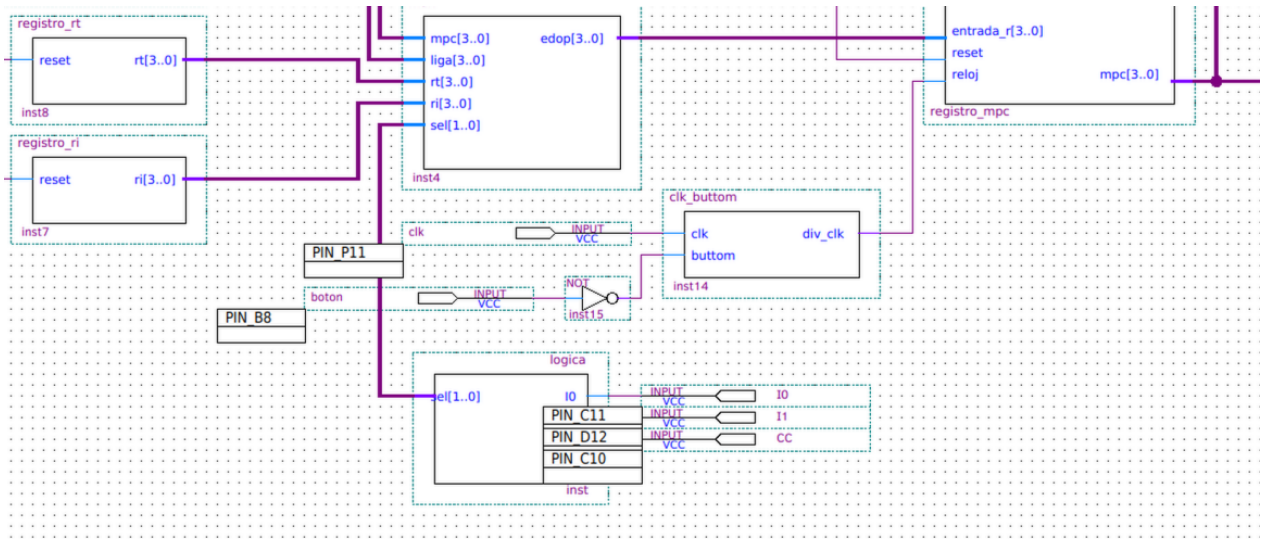


Figura 2: Interconexión final del multiplexor, los registros, la lógica interna y el contador de microprograma.

En la Figura[3] se observa con mayor detalle la realimentación entre el registro mPC y el

incrementador. El valor incrementado entra a la primera opción del multiplexor y constituye el recorrido natural del microprograma cuando se selecciona el paso continuo.

Es importante consideara que el único registro que se actualiza en el flanco ascendente del reloj es mPC, debido a esto, el bloque se debe de colocar después del multiplexor y no antes, ya que de lo contrario se perdería la información de la microdirección presente.

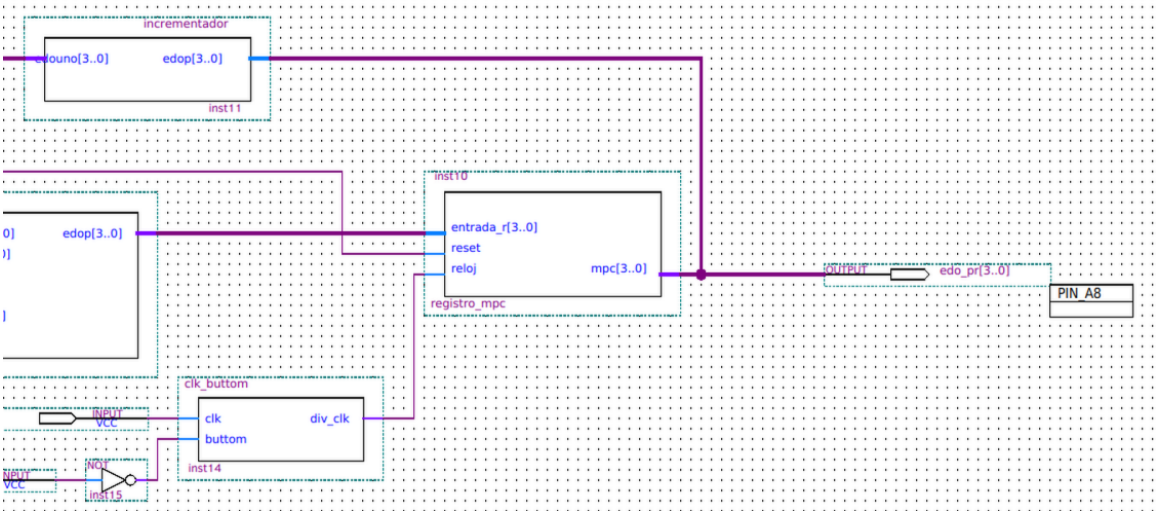


Figura 3: Camino de realimentación para el paso continuo $mPC + 1$.

2.3. Módulos implementados en VHDL

Contador de microprograma e incrementador. El módulo `registro_mpc` almacena la dirección que entrega el multiplexor. El reinicio es asíncrono y activo en bajo, por lo que `reset='0'` fuerza la dirección `0000`. El incrementador calcula de forma combinacional la dirección consecutiva de cuatro bits.

Listing 1: Registro del contador de microprograma.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity registro_mpc is
7      Port ( reloj : in std_logic;
8             reset : in std_logic;
9                entrada_r: in std_logic_vector(3 downto 0);
10            mpc : out std_logic_vector (3 downto 0)
11          );
12 end registro_mpc;
13
14 architecture Behavioral of registro_mpc is
15
16 signal internal_value: std_logic_vector (3 downto 0) := B"0000";
17 begin
```

```
    process(reloj, reset)
    begin
        if reset = '0' then
            internal_value <= B"0000";
        elsif rising_edge(reloj) then
            internal_value <= entrada_r;
        end if;
    end process;

    process(internal_value)
    begin
        mpc <= internal_value;
    end process;

end Behavioral;
```

Listing 2: Incrementador que genera $mPC + 1$.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity incrementador is
    Port ( edop: in std_logic_vector (3 downto 0);
           edouno : out std_logic_vector (3 downto 0)
         );
end incrementador;

architecture Behavioral of incrementador is
begin

    process(edop)
    begin

        edouno <= edop + 1;

    end process;

end Behavioral;
```

Multiplexor y lógica interna. El multiplexor `mux` concentra las cuatro fuentes de dirección. La lógica combinacional convierte los bits $I_1 I_0$ y la condición `cc` en el selector de dos bits, de acuerdo con la Tabla[1].

Listing 3: Multiplexor de cuatro fuentes para la siguiente microdirección.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity mux is
    Port ( mpc : in std_logic_vector (3 downto 0);
           liga : in std_logic_vector (3 downto 0);
           rt : in std_logic_vector (3 downto 0);
           ri : in std_logic_vector (3 downto 0);
           sel: in std_logic_vector (1 downto 0);
           edop: out std_logic_vector (3 downto 0)
         );
```

```
end mux;

architecture Behavioral of mux is
begin

    process(mpc, liga, rt, ri, sel)
    begin
        if sel = "00" then
            edop <= mpc;
        elsif sel = "01" then
            edop <= rt;
        elsif sel = "10" then
            edop <= ri;
        elsif sel = "11" then
            edop <= liga;
        end if;
    end process;

end Behavioral;
```

Listing 4: Lógica interna que implementa los cuatro tipos de microinstrucción.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity logica is
    Port ( I0: in std_logic;
           I1: in std_logic;
           cc: in std_logic;          -- Condición de control/prueba
           sel: out std_logic_vector (1 downto 0) -- Salida del selector de modo
         );
end logica;

architecture Behavioral of logica is
signal I_sel : std_logic_vector(1 downto 0);
begin
    I_sel <= I1 & I0;
    process(I_sel, cc)
    begin
        sel <= "00";
        case I_sel is

            when "00" =>
                sel <= "00";
            when "01" =>
                if cc = '0' then
                    sel <= "11"; -- Salto Condicional (selector=11 si cc=0)
                else -- cc = '1'
                    sel <= "00"; -- Paso Continuo (selector=00 si cc=1)
                end if;
            when "10" =>
                sel <= "01";
            when "11" =>
                if cc = '0' then
                    sel <= "10"; -- Salto por Interrupción (selector=10 si cc=0)
                else -- cc = '1'
                    sel <= "00"; -- Paso Continuo (selector=00 si cc=1)
                end if;

            when others =>
                sel <= "00";
```

```
        end case;
    end process;

end Behavioral;
```

Registros de dirección. Los registros de liga, transformación e interrupción entregan valores constantes después de liberar el reinicio: `1000`, `1010` y `1100`, respectivamente. Los registros implementados cumplen la función de almacenar las direcciones específicas para cada tipo de microinstrucción.

Listing 5: Registro que entrega la dirección de liga.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity registro_liga is
    Port ( reset : in std_logic;
           liga : out std_logic_vector (3 downto 0)
         );
end registro_liga;

architecture Behavioral of registro_liga is

signal internal_value: std_logic_vector (3 downto 0) := B"0000";
begin
    process(reset)
    begin
        if reset = '0' then
            internal_value <= B"0000";
        else
            internal_value <= B"1000";
        end if;
    end process;

    process(internal_value)
    begin
        liga <= internal_value;
    end process;

end Behavioral;
```

Listing 6: Registro de dirección de transformación.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity registro_rt is
    Port ( reset : in std_logic;
           rt : out std_logic_vector (3 downto 0)
         );
end registro_rt;

architecture Behavioral of registro_rt is

signal internal_value: std_logic_vector (3 downto 0) := B"0000";
```

```
begin
    process(reset)
    begin
        if reset = '0' then
            internal_value <= B"0000";
        else
            internal_value <= B"1010";
        end if;
    end process;

    process(internal_value)
    begin
        rt <= internal_value;
    end process;

end Behavioral;
```

Listing 7: Registro de dirección de interrupción.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity registro_ri is
    Port ( reset : in std_logic;
           ri : out std_logic_vector (3 downto 0)
         );
end registro_ri;

architecture Behavioral of registro_ri is

signal internal_value: std_logic_vector (3 downto 0) := B"0000";
begin
    process(reset)
    begin
        if reset = '0' then
            internal_value <= B"0000";
        else
            internal_value <= B"1100";
        end if;
    end process;

    process(internal_value)
    begin
        ri <= internal_value;
    end process;

end Behavioral;
```

Reloj por botón. El bloque `clk_buttom` genera un pulso lento y controlable a partir del reloj de la FPGA y del botón. Este bloque a su vez implemente un divisor de frecuencia para que el pulso de reloj sea lo suficientemente lento y permita observar los cambios de microdirección en la tarjeta.

Listing 8: Generación de un pulso de reloj controlado por botón.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity clk_buttom is
7      port(
8          clk : in std_logic;
9          buttom : in std_logic;
10         div_clk : out std_logic
11     );
12 end clk_buttom;
13
14 architecture Behavioral of clk_buttom is
15
16 CONSTANT STATE_START: std_logic_vector(1 downto 0) := "00";
17 CONSTANT STATE_COUNT: std_logic_vector(1 downto 0) := "01";
18 CONSTANT STATE_DONE: std_logic_vector(1 downto 0) := "10";
19
20 CONSTANT MAX_CLK_COUNT: std_logic_vector(27 downto 0) := X"17D7840"; -- 25 millones de
       pulsos
21
22 signal state: std_logic_vector(1 downto 0) := "00";
23 signal cuenta: std_logic_vector(27 downto 0) := X"0000000";
24
25 begin
26     process(clk, buttom)
27     begin
28         if rising_edge(clk) then
29             case state is
30                 WHEN STATE_START =>
31                     if buttom = '1' then
32                         state <= STATE_COUNT;
33                     end if;
34                 WHEN STATE_COUNT =>
35                     if cuenta >= MAX_CLK_COUNT then
36                         state <= STATE_DONE;
37                     end if;
38                 WHEN STATE_DONE =>
39                     state <= STATE_START;
40                 WHEN others =>
41                     state <= STATE_START;
42             end case;
43         end if;
44     end process;
45
46     process(clk)
47     begin
48         if rising_edge(clk) then
49             if state = STATE_COUNT then
50                 cuenta <= cuenta + 1;
51             else
52                 cuenta <= X"0000000";
53             end if;
54         end if;
55     end process;
56
57     process(state)
58     begin
59         if state = STATE_COUNT then
60             div_clk <= '1';
61         else
62             div_clk <= '0';
63         end if;
64     end process;
```

```
65          end Behavioral;
```

3. Simulación

La simulación se realizó con una señal de reloj periódica, reinicio activo en bajo y diferentes combinaciones de I_1, I_0 y CC. La señal `edo_pr` muestra el contenido del contador de microprograma. Debido a que mPC es un registro, cada dirección se actualiza en el flanco ascendente del reloj y permanece estable entre flancos.

En la Figura[4] se observan los cuatro valores importantes del diseño (I_0, I_1, CC, edo_pr). Además de los avances consecutivos como `1000`→`1001`→`1010`, aparecen `1000` por la liga, `1010` por el registro de transformación y `1100` por el registro de interrupción. Esto permite reconocer la fuente seleccionada a partir de la salida del contador.

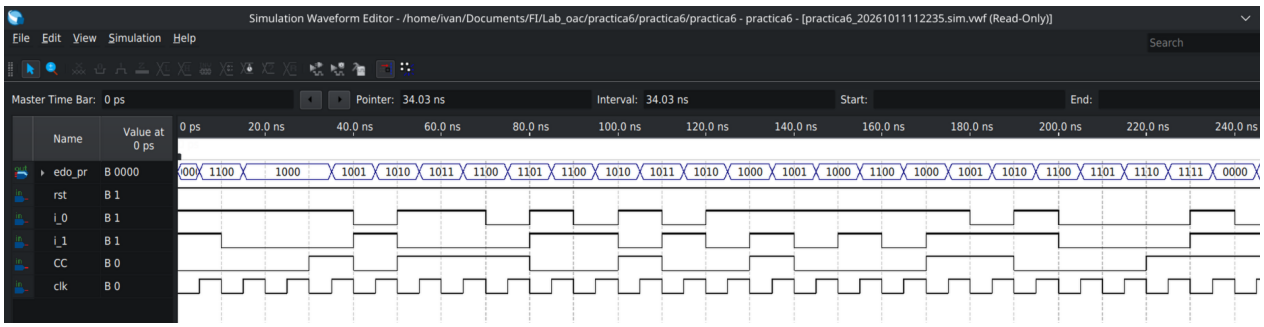


Figura 4: Primera parte de la simulación funcional del secuenciador.

La Figura[5] verifica la respuesta de la lógica interna al cambiar las entradas durante la simulación. Cuando $I_1I_0 = 01$, el valor de CC decide entre el salto a la liga y el paso continuo. Para $I_1I_0 = 10$, la salida toma directamente el valor del registro RT. Para $I_1I_0 = 11$, sólo se usa el registro RI cuando $CC = 0$.

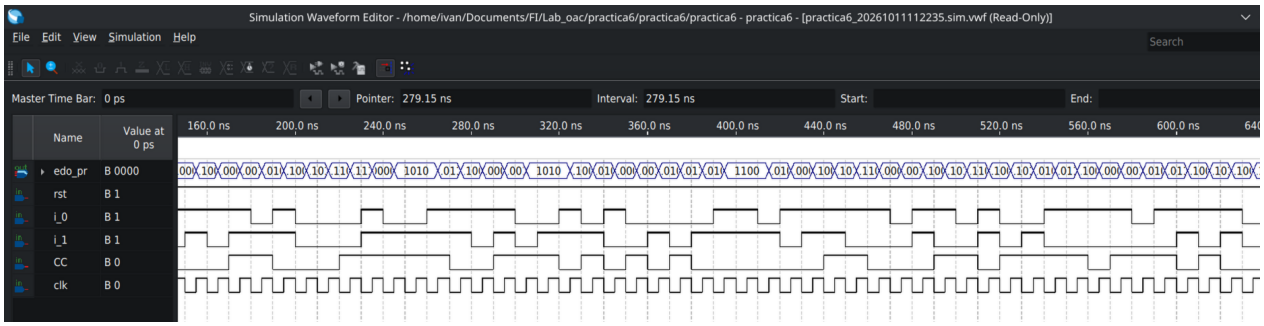


Figura 5: Cambios de microdirección al modificar I_1, I_0 y CC.

En la parte final, mostrada en la Figura[6], se repiten los modos de operación con nuevas combinaciones de entrada. La señal `edo_pr` no depende directamente de los cambios intermedios de las entradas, sino del valor que la lógica seleccione al siguiente flanco de reloj. Con ello se comprueba la operación síncrona del secuenciador alternativo.

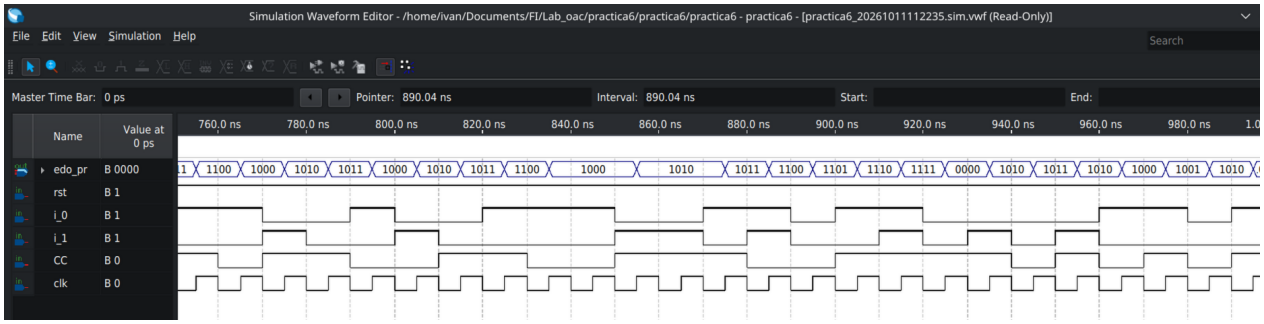
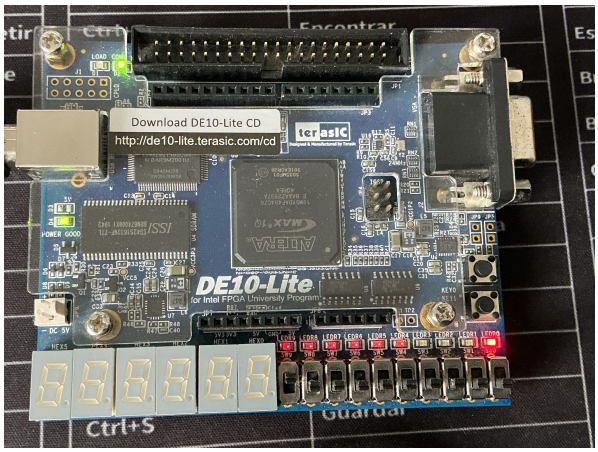


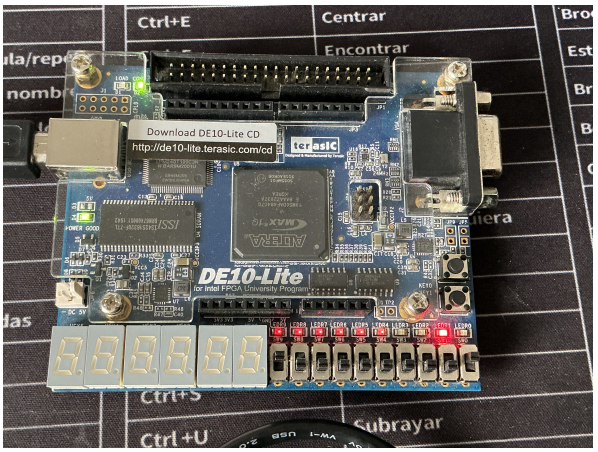
Figura 6: Parte final de la simulación, con recorridos continuos y saltos.

4. Resultados en la tarjeta FPGA

El diseño se implementó en la tarjeta de desarrollo DE10-Lite. Las fotografías muestran distintas combinaciones de $I_1 I_0 CC$ y los LEDs asociados a la microdirección presente. Cada combinación activa la fuente indicada en la Tabla[1], por lo que se pueden distinguir visualmente los modos continuo, condicional, de transformación y de interrupción.

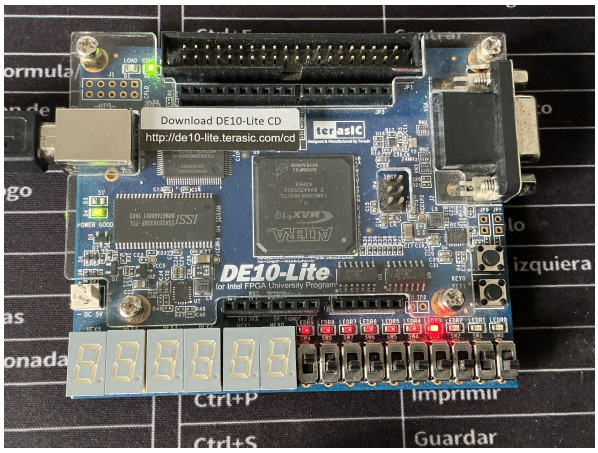


(a) Combinación `000`: paso continuo.

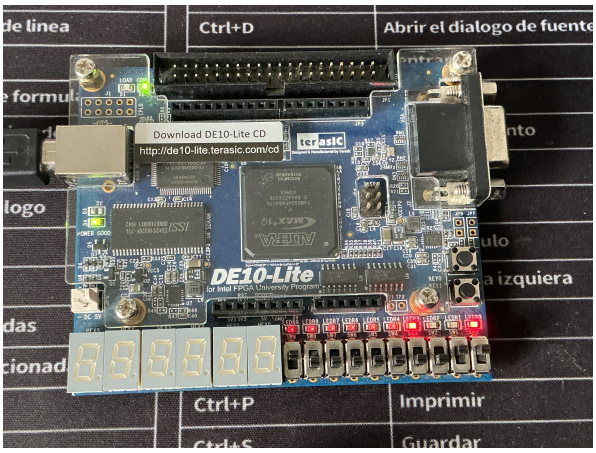


(b) Combinación `001`: paso continuo.

Figura 7: Comprobación física del modo $I_1 I_0 = 00$.

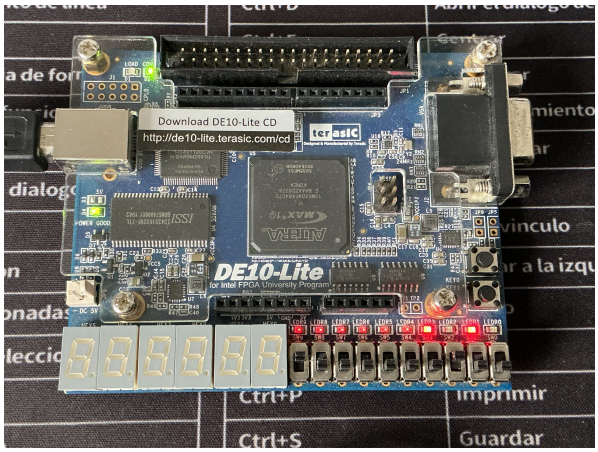


(a) `010`: salto condicional a la liga.

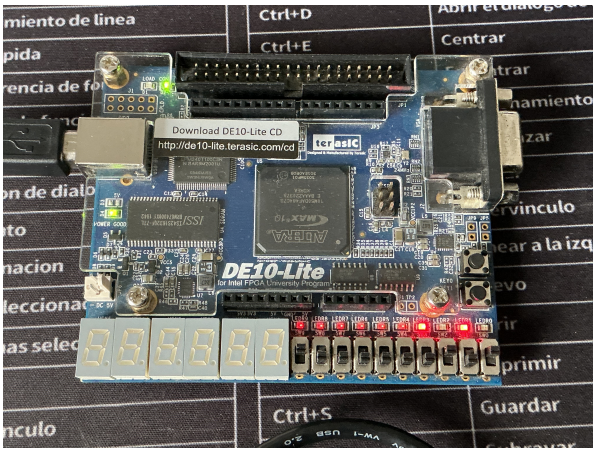


(b) `011`: avance continuo.

Figura 8: Comprobación del salto condicionado por CC para $I_1 I_0 = 01$.

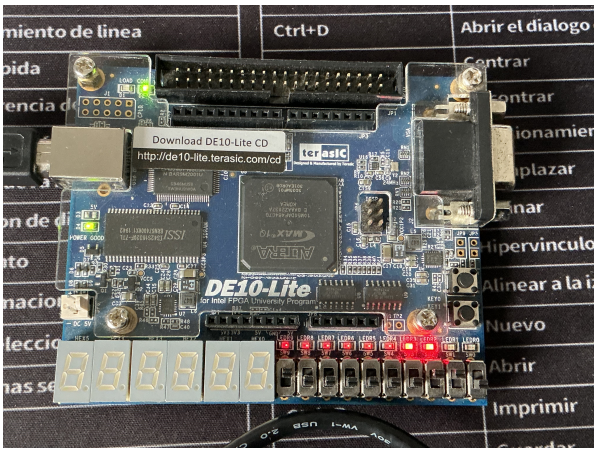


(a) `100`: salto de transformación.

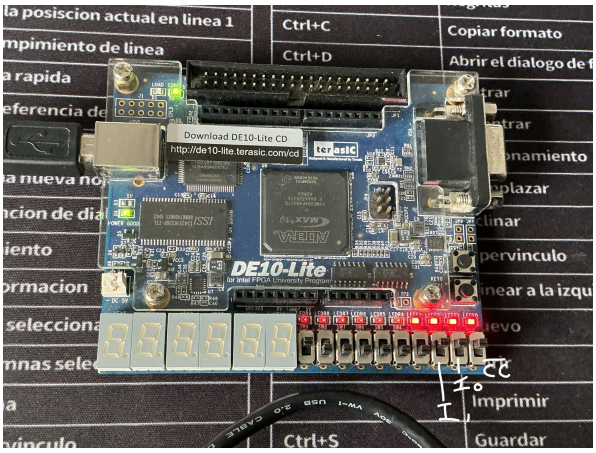


(b) `101`: salto de transformación.

Figura 9: Comprobación del registro RT para $I_1 I_0 = 10$.



(a) `110`: salto de interrupción.



(b) `111`: avance continuo.

Figura 10: Comprobación del modo de interrupción para $I_1 I_0 = 11$.

La implementación física confirma que la siguiente microdirección sólo se registra en el pulso de reloj y que la lógica interna selecciona correctamente una de las cuatro fuentes. También se comprobó que el reinicio activo en bajo devuelve el contador de microprograma a `0000`, lo que permite iniciar cualquier secuencia desde una dirección conocida.

5. Conclusión

Percival Ulises Espinoza Matamoros. A partir de la realización de la práctica logró implementar un secuenciador básico alternativo, donde al realizar la implementación ayudo a comprender el funcionamiento de este tipo de secuenciador. El secuenciador alternativo permite seleccionar la siguiente microdirección a partir de cuatro fuentes, dependiendo de los bits de microinstrucción y de la condición de control. Por medio de la simulación así como de la implementación física se pudo comprobar y verificar el funcionamiento del secuenciador, así como la relación entre las microinstrucciones y la ruta de control.

Oscar Iván Montiel Juárez. El desarrollo de esta práctica permitió conocer la importancia del contador de microprograma y de la lógica de selección dentro de un procesador. Al conectar el incrementador, los registros de liga, transformación e interrupción con el multiplexor se pudo comprobar que una sola arquitectura puede generar distintos recorridos dependiendo de los bits de microinstrucción y de la condición de control. La simulación y la tarjeta permitieron verificar cada caso de manera visual.

Amy Valentina Veraza García. La práctica ayudó a reforzar la relación entre las microinstrucciones y el funcionamiento de la ruta de control. Al probar el circuito en la DE10-Lite fue posible observar los cambios de dirección en cada pulso de reloj y comprobar que el valor de CC sólo afecta a los tipos de salto que requieren una decisión.